

ZFS Reblock Worker

Safe ZFS block rewriting at production scale

Version 4.4.14

September 24, 2026

Author: Hernán Dario Nacimiento

github.com/hnacimiento/zfs-reblock-worker

Repository and test evidence: see README.md (bilingual), docs/testing/ (Spanish) and CHANGELOG.md (Spanish)

Executive summary

ZFS Reblock Worker is a Bash worker for TrueNAS SCALE / Linux^[25] that rewrites large files inside their own ZFS dataset so their blocks adopt the recordsize the dataset already has configured. Changing a dataset's recordsize converts nothing by itself: it only affects new writes^[1]. This worker does the real work, file by file, without risking the data.

Every file is copied to a temp file in the same directory, verified with a cryptographic hash against the source, and only then replaces the original via an atomic rename within the same dataset — it never truncates or overwrites the original in place. The process can die at any instruction, at any moment, even from a power loss, and the next run safely reconstructs what it was doing.

Current status: version 4.4.14, with sixteen rounds of testing — the vast majority against real ZFS, not simulated — and two full reblocks in real production (Musicales and Videos/Series, 2.1 TiB, 238 files, 0 failures). It survived two real machine reboots mid-copy without losing a single byte.

Why it exists

An administrator changes a dataset's recordsize from its default to 1M. That change only affects new writes: files already there keep the block size they were written with. For the change to take real effect, large files need to be rewritten — and rewriting them must not change their identity: mtime and atime have to keep their original timestamps.

```
ORIGINAL --rsync -aAXS--> TEMP (same directory)
                        |
                    full verification
                        |
                    atomic rename
                        |
                    ORIGINAL
```

Safety contract

These eleven properties are invariants: there is no command-line option that disables them.

1. Snapshot inventory. A job processes exactly the universe detected when it started.
2. Copy to a temp file in the same directory, guaranteeing the commit is a rename within the same dataset.
3. Source identity verified (size + inode + mtime) before copying, during, and again right before the commit.
4. Cryptographic hash of the source against the temp file before the commit (default BLAKE2b^[17], cascading to BLAKE3^[18] and SHA-256^[19]).
5. atime and mtime captured^[8] before reading the file, and explicitly restored.
6. Commit only by rename.^[6] The original is never truncated or overwritten in place.
7. State persisted before and after every relevant boundary.
8. Conservative recovery: an orphaned temp file goes to quarantine, never promoted or deleted.
9. Rollback anchor: a hardlink^[7] to the original inode keeps the data reachable until post-commit verification passes.
10. The dataset's recordsize is never modified.
11. ACL and xattr verified with whatever tool the dataset requires; without a verifier, the file is excluded from the inventory.

Design premise: the process can die at any instruction, at any moment, and the next run must be able to safely reconstruct what it was doing and continue.

How well-tested this is

Sixteen rounds of testing, the vast majority against real ZFS on the same reference TrueNAS box:

- Two real machine reboots mid-copy (one planned, one from a real power outage) — the original file came out intact both times.
- kill -9, two real workers running simultaneously on the same job, and a real Ctrl+C — none of them compromised the original.
- All of Musicales and all of Videos/Series reblocked in real production: 2.1 TiB, 238 files, 0 failures, verified with hash, ACL, mtime and zdb.
- A full live interactive console session found and fixed three real bugs; the production-scale round found and fixed five more — none of the eight ever compromised the integrity of the original.

Full round-by-round detail, with raw evidence, lives in the repository's docs/testing/ (Spanish). What still hasn't been tested: the rest of the dataset (a rollout decision, not a test) and non-trivial POSIX ACL / real atime=on against a production dataset (only verified in a lab).

Adaptive Resource Governor

Before touching the first file, the worker observes the server for a few seconds and establishes a baseline. While it works, it computes a health score from CPU, I/O wait, load and memory, and how much all of that deviates from the baseline. I/O wait weighs more than CPU, because in a reblock the delicate resource is storage.

Three recent improvements, found and fixed against real hardware:

- The memory reading credits back the ZFS ARC^[2] that would be reclaimable without trouble — without this adjustment, a healthy host with high ARC read as if it had a real problem.
- After a protective pause, the governor waits a short cooldown before raising bandwidth back up, so it doesn't collide with ZFS's next periodic sync burst.^[5]
- The pressure score credits evidence that the disk is still delivering real bytes — the same principle TCP BBR^[15] and adaptive concurrency limiters^[16] use: a disk busy being useful isn't the same as one busy and stuck.

With --adaptive off the governor stops acting but keeps measuring; operator authority is always absolute: a manual BW ceiling can never be exceeded by the automatic system.

Basic usage

Wizard, no arguments

```
./zfs-reblock-worker.sh
```

Detects the state of previous jobs and only offers the options that make sense for that situation. Includes a directory browser.

Command line, for automation

```
./zfs-reblock-worker.sh /mnt/toolbox/media --bwlimit 20M --adaptive guarded
./zfs-reblock-worker.sh /mnt/toolbox/media --dry-run --dry-sample 5
./zfs-reblock-worker.sh --check
./zfs-reblock-worker.sh --status --json
```

See --help in the script itself for the full list of options.

Dependencies

Type	Tools	If missing
Critical	bash awk find sort grep sed rsync ^[24] flock ^[9] zfs sync sha256sum id kill, plus UID 0	The worker won't start
Per acltype ^[3]	nfs4xdr_getfacl+jq (nfsv4) / getfacl+getfattr (posix) / getfattr (off)	That dataset falls out of the inventory
Optional	zdb ^[4] nice ^[12] renice ionice ^[11] whiptail dialog fzf b2sum b3sum timeout	Controlled, announced degradation, never a silent failure

Exit codes

Code	Meaning
0	Everything correct, or job paused safely
12	The job finished with at least one file in FAILED
13	The job stopped early (space, operator, governor)
20	A critical dependency is missing, or no dataset is verifiable
23	The dataset's recordsize isn't the expected one

Project origin

The current worker is the result of sixteen rounds of testing and fixes on top of a design that started, in September 2026, as a point-by-point architecture discussion built on an initial prototype (V2) that only verified size and had no real checkpoint. That full history — the original prototype, intermediate versions, and the design discussion that produced V3 — is kept in the repository's `legacy/` folder, outside the main tree, as an honest record of where the project came from without mixing it into the current state.

Identity and verification of this document

This document is published digitally signed (PAdES)^[22] with the personal certificate^[21] of Hernán Dario Nacimiento, published under `certificate/` in the repository together with its SHA-256 fingerprint. The `.docx/.htm` version also ships its own `.sha256` file. Any copy whose signature doesn't validate, or whose hash doesn't match, isn't the original. The definitive source of truth is the project's GitHub repository: every version is anchored to a commit with its own hash, independently verifiable.

© Hernán Dario Nacimiento. Document generated from the project's real, verified state as of the date on the cover.

References

1. [↑ — zfsprops.7 — OpenZFS documentation](#). [Archived version](#) · Accessed September 24, 2026.
2. [↑ — Module Parameters — OpenZFS documentation \(zfs arc min\)](#). [Archived version](#) · Accessed September 24, 2026.
3. [↑ — zfsprops.7 — OpenZFS documentation \(acttype\)](#). [Archived version](#) · Accessed September 24, 2026.
4. [↑ — zdb.8 — OpenZFS documentation](#). [Archived version](#) · Accessed September 24, 2026.
5. [↑ — Module Parameters — OpenZFS documentation \(zfs txg timeout\)](#). [Archived version](#) · Accessed September 24, 2026.
6. [↑ — rename\(2\) — Linux manual page](#). [Archived version](#) · Accessed September 24, 2026.
7. [↑ — link\(2\) — Linux manual page](#). [Archived version](#) · Accessed September 24, 2026.
8. [↑ — zfsprops.7 — OpenZFS documentation \(atime\)](#). [Archived version](#) · Accessed September 24, 2026.
9. [↑ — flock\(2\) — Linux manual page](#). [Archived version](#) · Accessed September 24, 2026.
11. [↑ — ionice\(1\) — Linux manual page](#). [Archived version](#) · Accessed September 24, 2026.
12. [↑ — sched\(7\) — Linux manual page \(nice value\)](#). [Archived version](#) · Accessed September 24, 2026.
15. [↑ — TCP BBR congestion control comes to GCP \(Google Cloud Blog\)](#). [Archived version](#) · Accessed September 24, 2026.
16. [↑ — Adaptive Concurrency — Envoy documentation](#). [Archived version](#) · Accessed September 24, 2026.
17. [↑ — RFC 7693: The BLAKE2 Cryptographic Hash and MAC](#). [Archived version](#) · Accessed September 24, 2026.
18. [↑ — BLAKE3 — official repository \(BLAKE3-team\)](#). [Archived version](#) · Accessed September 24, 2026.
19. [↑ — FIPS PUB 180-4: Secure Hash Standard \(NIST\)](#). [Archived version](#) · Accessed September 24, 2026.
21. [↑ — RFC 5280: Internet X.509 PKI Certificate and CRL Profile](#). [Archived version](#) · Accessed September 24, 2026.
22. [↑ — ETSI EN 319 142-1: PAdES digital signatures](#). [Archived version](#) · Accessed September 24, 2026.
24. [↑ — rsync\(1\) man page \(rsync / Samba project\)](#). [Archived version](#) · Accessed September 24, 2026.
25. [↑ — First Official RELEASE of TrueNAS on Linux! \(iXsystems\)](#). [Archived version](#) · Accessed September 24, 2026.